

Software Architecture, Final Exam Study Guide

Part 1: Multiple-choice questions

Part 2: Short answer questions (e.g., “What is Software Architecture?”)

Part 3: Modeling Questions: In this section, you may be required to draw or explain diagrams.

Understanding Software Architecture (Main Book - Essential Software Architecture)

1. What is Software Architecture?

Arkitektura e softuerit të një programi ose sistemi kompjuterik është struktura ose strukturat e sistemit, të cilat përfshijnë elementet e softuerit, vetitë e jashtme të dukshme të atyre elementeve dhe / ndërmjet tyre.

2. What Does a Software Architect Do?

- Merr vendimet kyçe të dizajnit
- E përafron arkitekturën me objektivat e biznesit
- Zgjedh teknologjitë dhe pattern-et
- Dokumenton dhe komunikon arkitekturën

3. Why is Software Architecture Important?

- **Arkitektura** përqendrohet në mënyrën se si elementët dhe komponentët kryesorë brenda një aplikacioni përdoren nga, ose ndërveprojnë me, elementët dhe komponentët e tjerë të rëndësishëm të aplikacionit.

4. Which are the four Architecture Model Views?

Logical View, Process View, Physical View, Development View.

5. What are the three distinct areas of nonfunctional requirements?

Technical constraints, Business constraints, Quality attributes.

Architectural Styles and Patterns(Book 2: Documenting Software Architectures)

1. Explain the terms “Architecture Style” and “Architecture Pattern”?

- Një model arkitekture “shpreh një skemë themelore të organizimit strukturor për sistemet softuerike.” Pattern = { context + problem + solution (+ rationale) }
- Një stil arkitekturor fokusohet në qasjen arkitekture, duke ofruar udhëzime më të lehta për të kuptuar kur një stil i caktuar mund ose nuk mund të jetë i dobishëm.
Style = { types of architectural elements + types of relationships + rules }

{Një Stil Arkitekturor  është dizajni i aplikacionit në nivelin më të lartë të abstraksionit.

Një Model (Pattern) Arkitekturor  është mënyra konkrete për të zbatuar një Stil Arkitekturor.}-from link(Moodle)

2. List three Categories of Styles?

Module Style, Component-and-Connector Style, Allocation Style.

3. What are some common architectural styles for the Module styles category and give a short description for one of the styles?

- Decomposition
 - Uses / Dependency
 - Generalization
 - Layered
 - Aspects
 - Data Model / View
- **Layered** – Modulet grupohen në shtresa horizontale ku çdo shtresë ofron shërbime vetëm për shtresën menjëherë mbi të dhe varet nga ajo poshtë saj; kjo ndarje thjeshton mirëmbajtjen, testimin dhe zëvendësimin e shtresave.
 - **Decomposition view** tregon se si ndahen përgjegjësitë nëpër module dhe nënmodule.
 - **User View** tregon se si modulet varen nga njëri-tjetri.

4. What are Components and Connectors, give an explanation for each of them?

Komponentet janë elementet që kanë njëfarë prezence në kohezgjatje, siq janë proceset, objektet, klientet, serveret dhe data stores.

Konektoret janë elemente që shërbejnë si rrugë e nderveprimit, siq janë lidhjet e komunikimit dhe protokolet, rrjedhat e informacionit dhe qasje në shared storage

Software Quality Attributes (Main Book - Essential Software Architecture)

1. What are Quality attribute requirements?

Janë qëllime jofunksionale të matshme që përcaktojnë sa mirë duhet të sillet sistemi, jo çfarë funksionesh duhet të kryejë. Ato konkretizojnë si shkallëzueshmëria, performanca, siguria, besueshmëria, mirëmbajtja etj., në deklarata të sakta dhe të testueshme që udhëheqin vendimet arkitekturore.

2. Which are General Software Quality Attributes(“-ilities”)?

shkallëzueshmërinë , sigurinë , performancën  dhe besueshmërinë .

3. “Authentication” and “Authorization” are security-related requirements. Give a short explanation for each of them.

- **Autentikimi**  – Procesi me të cilin sistemi verifikon identitetin e një përdoruesi apo aplikacioni (p.sh. me fjalëkalim, token, certifikatë).
- **Autorizimi**  – Procesi që përcakton çfarë mund të bëjë një përdorues apo aplikacion i autentikuar: lexim, shkrim, administrim, etj.

 Pra, autentikimi përgjigjet “Kush je?”, ndërsa autorizimi “Çfarë ke të drejtë të bësh?”.

4. Give a short description of “Performance” as one of the quality requirements, and illustrate with an example from your course project.

Performance – mat sa shpejt sistemi kthen përgjigje dhe përpunon ngarkesën.

Shembull (Sistemi i Aplikimit për Punë):

Lista e vendeve të lira duhet të ngarkohet ≤ 1 sek në 95 % të rasteve, edhe me 200 përdorues njëkohësisht, duke përdorur caching dhe indeksim DB.

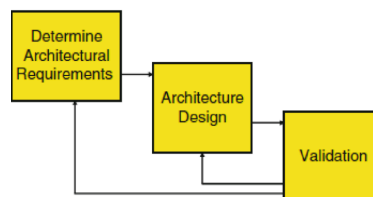
***A Software Architecture Process* (Main Book: Essential Software Architecture)**

1. Which are the three steps of the architectural design process, illustrated with a diagram? sti

Përcaktoni kërkesat e arkitekturës: Kjo përfshin krijimin e një deklarate ose modeli të kërkesave që do të nxisin dizajnin e arkitekturës.

Dizajni i arkitekturës: Kjo përfshin përcaktimin e strukturës dhe përgjegjësi të komponentëve që do të përbëjnë arkitekturën.

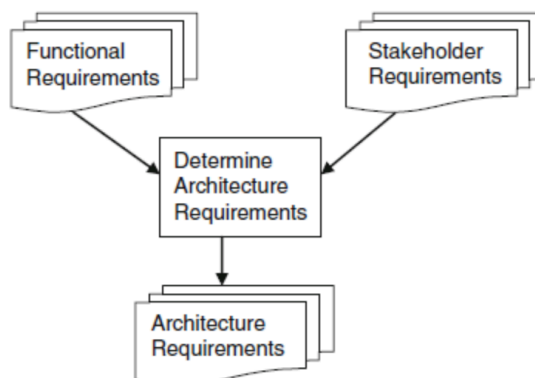
Validimi: Kjo përfshin "testimin" e arkitekturës, zakonisht duke kaluar nëpër dizajn, kundrejt kërkesave ekzistuese dhe çdo kërkesë të njohur ose të mundshme në të ardhmen.



2. Which are the Inputs and outputs that determine architecture requirements? Illustrate with a diagram.

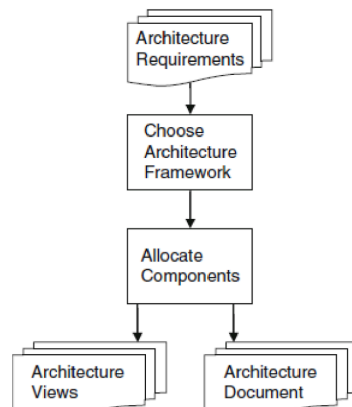
Input janë: Kërkesat funksionale (Functional Requirements), Kërkesat jofunksionale (Non-functional Requirements), Dokumentet nga palët e interesit (e.g., user stories, business goals, constraints).

Output është dokumenti i kërkesave të arkitekturës (Architecture Requirements Specification). Ky dokument përmbledh të gjitha kërkesat që do të ndikojnë në vendimet arkitekturore të sistemit.



3. What are the Inputs and outputs of architecture design? Illustrated with a diagram?

Inputet ne dizajn jane kerkesat e arkitektures. Kurse **outputi** eshte nje set i pamjeve te arkitektures te cilat kapin dizajnin e arkitektures dhe dokumentin e dizajnit qe shpjegon dizajnin.



4. What is the aim of the Validation phase during the architecture process?

Gjatë procesit të arkitekturës, qëllimi i fazës së Validimit është të rrisë besimin e ekipit të projektimit duke ju treguar se arkitektura është e përshtatshme për qëllimin e tyre.

Architecture in Agile Projects (Book 3: Software Architecture in Practice & Slides: Week 7 - Architecture in Agile Projects)

1. List some of the main principles from the Agile Manifesto. (Minimum four).

Kënaqësia e klientit përmes dorëzimit të shpejtë të softuerit të dobishëm 🚀

Mirëpritja e kërkesave që ndryshojnë edhe në fazat e vona të zhvillimit

Bashkëpunim i ngushtë, çdo ditë, mes biznesit dhe zhvilluesve

Softueri i punës shpërndahet shpesh (në javë dhe jo muaj)

Softueri i punës është matësi kryesor i progresit

Ekipet vetëorganizuese

Biseda ballë për ballë është forma më e mirë e komunikimit (bashkë)

2. List some AGILE Methods?(Minimum three).

Scrum, Kanban, Extreme Programming (XP), Crystal Clear, Lean, Feature-Driven

Development (FDD), Dynamic Systems Development Method (DSDM), Adaptive Software

Development (ASD), Agile Unified Process (AUP).

3. Which are the three core roles in the SCRUM method, and give a short description for each of them?

–Product Owner

The Product Owner represents the stakeholders and is the voice of the customer.

–Development Team

The Development Team is responsible for delivering potentially shippable increments (PSIs) of product at the end of each Sprint (the Sprint Goal).

–Scrum Master

Scrum is facilitated by a Scrum Master, who is accountable for removing impediments to the ability of the team to deliver the product goals and deliverables.

The Scrum Master is not a traditional team lead or project manager, but acts as a buffer between the team and any distracting influences.

The Scrum Master ensures that the Scrum process is used as intended.

4. What are some characteristics of AGILE in the Software development life cycle (SDLC)?

-Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations. Each iteration typically lasts from about one to three weeks. Every iteration involves cross functional teams working simultaneously on various areas like planning, requirements analysis, design, coding, unit testing, and acceptance testing. At the end of the iteration a working product is displayed to the customer and important stakeholders.

Understand Monolithic & Microservice Architecture, Docker and Docker Containers

1. Compare containers and virtual machines regarding resource efficiency and isolation.?

Konteinerët janë më të lehtë dhe më efikas në përdorim të burimeve se makinat virtuale. Ata mund të përmbajnë aplikacione të shumta dhe varësi të tyre në një hapësirë të vogël, duke ndarë kernelin e sistemit operativ. Ndërsa makinat virtuale kërkojnë shumë hapësirë për t'u instaluar dhe menaxhuar, duke u përballur me më shumë ngarkesa në burime.

2. What are the main benefits of using monolithic architecture for a software application?

- **E Thjeshtë për Të Zhvilluar:** Në fillim të një projekti, është shumë më e thjeshtë të punosh me arkitekturën monolitike, pasi gjithçka është e centralizuar në një aplikacion të vetëm.
- **E Thjeshtë për Të Testuar:** Për shembull, mund të realizosh teste end-to-end thjesht duke nisur aplikacionin dhe testuar ndërfaqen e përdoruesit (UI) me Selenium.
- **E Thjeshtë për Të Implementuar:** Mjafton të kopjosh aplikacionin e paketuar në një server dhe të fillosh ta përdorësh.
- **E Thjeshtë për Të Shkallëzuar Horizontal:** Mund të shkallëzosh aplikacionin duke shtuar kopje të shumta të tij pas një balancuesi ngarkese (load balancer).

3. What are the main benefits of using microservice architecture for a software application?

Përfitimet e Arkitekturës me Mikroshërbime 

- Mundëson dorëzim dhe vendosje të vazhdueshme për aplikacione të mëdha e komplekse.
- Testueshmëri më e mirë — shërbimet janë të vogla dhe testohen më shpejt.
- Deploy më i lehtë — çdo shërbim mund të vendoset në mënyrë të pavarur.
- Çdo mikroshërbim është relativisht i vogël, prandaj: 🖱️ Më i lehtë për t'u kuptuar e mirëmbajtur nga zhvilluesit
- Mikroshërbimet eliminojnë çdo angazhim afatgjatë ndaj një stack-u teknologjik.

4. List and describe three advantages of using Docker containers over traditional virtual machines?

1. Lehtësi & kursim burimesh ⚡

Kontejnerët ndajnë kernel-in e hostit, prandaj kërkojnë shumë më pak RAM/CPU dhe nisen në sekonda.

2. Koherencë mjedisi nga dev në prod

Imazhi Docker paketon kodin + varësitë, duke siguruar që “funksionon tek unë” = “funksionon në server”.

3. DevOps & CI/CD miqësor

Integrim i drejtpërdrejtë me pipelines dhe orkestrues (p.sh. Kubernetes), që lejon ndërtim, testim dhe vendosje të shpejtë e të pavarur të mikroshërbimeve.

5. Describe the architecture of Docker. What are Docker images, Docker containers, and Docker registries?

Docker është një platformë me arkitekturë klient-server që lejon paketimin dhe ekzekutimin e aplikacioneve në një mjedis të izoluar të quajtur *container*. Ai përbëhet nga **Docker Engine**, klienti (CLI) dhe një REST API për komunikim. **Docker images** janë paketa të gatshme që përmbajnë gjithçka të nevojshme për të ekzekutuar një aplikacion. Nga këto krijohen **Docker containers**, të cilët janë instanca të lehta që ofrojnë mjedis të qëndrueshëm dhe të njëjtë në çdo sistem. Këto imazhe ruhen dhe shpërndahen përmes **Docker registries** si Docker Hub. Docker ofron izolim, efikasitet dhe përputhshmëri, duke e bërë shumë të përshtatshëm për mikroshërbime dhe praktika DevOps.

UML and Documenting Software Architecture

(Main Book: Essential Software Architecture & Book 3: Software Architecture in Practice)

1. What is Unified Modeling Language (UML)?

Unified Modeling Language (UML) është një gjuhë e standardizuar e modelimit vizual që përdoret në inxhinierinë e softuerit për të dizajnuar, vizualizuar dhe dokumentuar sisteme softuerike. Ai siguron një sërë shënimesh dhe teknikash grafike për të përfaqësuar aspekte të ndryshme të një sistemi, duke përfshirë strukturën, sjelljen dhe ndërveprimet e tij.

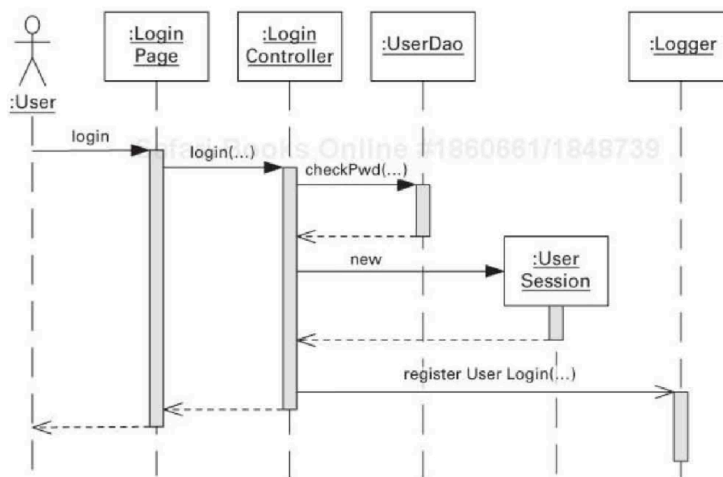
2. What defines the structure diagrams?

Diagramet e strukturës përcaktojnë arkitekturën statike të një modeli dhe konkretisht janë: Class Diagrams, Component Diagrams, Package Diagrams, Deployment Diagrams, Object Diagrams, Composite Structure Diagrams.

3. What defines the behavior diagrams?

Diagramet e sjelljes tregojnë ndërveprimet dhe ndryshimet e gjendjes që ndodhin kur elementet në model ekzekutohen. Diagrame të tilla janë: Activity Diagrams, Communication Diagrams, Sequence Diagrams, State Machine Diagrams, Interaction Overview Diagrams, Timing Diagrams, Use Case Diagrams.

4. Give a simple example of the UML sequence diagram? (illustrate with diagram)



5. What are the three main categories of notations for Architecture Documentation?

Structural Notations

Behavioral Notations

Textual or Descriptive Notations

6. What are the three steps of choosing the Views?

Per të zgjedhur një pamje, ndjekën 3 hapat: 1. Build a stakeholder/view table; 2. Combine views; 3. Prioritize and stage.

Designing an Architecture

(Book 3: Software Architecture in Practice & Slides: Week 11 and 12 - Designing an Architecture)

1. Which method should be used to satisfy the architecture design's quality and functional requirements?

Metoda e quajtur Attribute Driven Design (ADD) perdoret per te hartuar nje arkitekture per te permbushur kerkesat e cilesise dhe kerkesat funksionale.

2. What are the five steps of the Attribute-Driven Design Method (ADD)?

- Krijimi i dizajnit të të dhënave
- Derivimi i një ose më shumë paraqitjeve të strukturës arkitekturore të sistemit
- Analiza e stileve alternative arkitekturore për të zgjedhur atë që i përshtatet më së miri kërkesave të klientit dhe atributëve të cilësisë
- Elaborimi i arkitekturës bazuar në stilin arkitekturor të përzgjedhur

3. What are the three key design strategies?

- Decomposition
- Designing to Architecturally Significant Requirements
- Generate and Test

Architecture Evaluation

(Book 3: Software Architecture in Practice & Slides: Week 11 and 12 - Architecture Evaluation)

1. Types of Architecture Evaluation

Teknik: Vlerësimi kundrejt atributëve të cilësisë së sistemit, p.sh. performanca, siguria dhe modifikueshmëria, Përshtatshmëria e vendimeve të projektimit, P.sh. Architecture Tradeoff Analysis Method (ATAM).

Ekonomik: Kompensimet më të mëdha në sistemet komplekse të mëdha zakonisht kanë të bëjnë me ekonominë, koston dhe përfitimet që lidhen me vendimet e projektimit arkitekturor, p.sh. Cost Benefit Analysis Method (CBAM).

2. Which method should be used to evaluate a Software Architecture?

Architecture Tradeoff Analysis Method (ATAM) është procesi që kryen vlerësimet e arkitekturës. ATAM është përdorur për më shumë se dy dekada për të vlerësuar arkitekturat e softuerëve të sistemeve të mëdha në fusha që variojnë nga automobilistikë tek ato financiare e deri te mbrojtja.

3. Which method has been used over the decade to evaluate the Software architecture?

Architecture Tradeoff Analysis Method (ATAM) është procesi që kryen vlerësimet e arkitekturës. ATAM është përdorur për më shumë se dy dekada për të vlerësuar arkitekturat e softuerëve të sistemeve të mëdha në fusha që variojnë nga automobilistikë tek ato financiare e deri te mbrojtja.

4. What three groups must the ATAM method participate and cooperate with during the evaluation?

ATAM kërkon pjesëmarrjen dhe bashkëpunimin e ndërsjellë të tre grupeve: Evaluation team, Project decision makers, Architecture stakeholders.

NOTE:

☐ Regarding the questions with diagrams, in the exam you may be required to draw or explain diagrams.

☐ These questions are only for the orientation during the study; the professor reserves the right to change or modify them.